TP Classification Binaire : Régression Logistique

de Boisvilliers Christopher

April 8, 2025

TP : Régression Logistique sur des Données Kaggle

Objectif

Construire un modèle de classification binaire avec une régression logistique, évaluer ses performances avec des métriques de classification, et tester des stratégies d'optimisation.

Bloc 1 : Chargement et exploration des données

Étapes :

1. Choisir un dataset avec une cible binaire, à faire valider.
2. Expliquer votre dataset.
3. Charger les données dans un notebook Jupyter.
4. Visualiser les premières lignes. (head)
5. Nettoyer les données : types, valeurs manquantes, encodages.
6. Explorer les distributions, describe, corrélations, outliers (boxplot).
7. Proposer de nouvelles features variables à partir de vos données et expliquer leurs impact.
8. Proposer une explication sur chacune de vos découvertes à la partie 6.
9. Proposer des graphiques sexy permettant d'expliquer certains élèments de vos données

Pour le point 7 j'attends de vous de comprendre assez bien votre jeu de donnée afin de proposer de nouvelle features…

Datasets recommandés :

- User vs bots classification
- Diabetes prediction dataset
- Water Quality
- Airline Passenger satisfaction

Ou proposez moi quelque chose ici:

- Kaggle Binary Classification Datasets

Bloc 2 : Préparation des données et sélection des variables

Étapes :

1. Encodage des variables qualitatives.
2. Standardiser les données (obligatoire pour régression logistique).
3. Calculer les VIF pour vérifier la multicolinéarité.
4. Enlever les variables redondantes ou inutiles.

Rappel : transformations utiles

Problème	Transformation courante
Asymétrie forte	log(x), sqrt(x)
Effet multiplicatif	log(x)
Outliers	RobustScaler, log(x)

Bloc 3 : Entraînement du modèle

Étapes avec vous valeurs scaller mais non selectionnées :

1. Split : `X_train_w`, `X_test_w`, `y_train_w`, `y_test_w`.
2. Entraîner une `LogisticRegression()`.
3. Afficher les coefficients et les p-values avec `statsmodels`.
4. Interpréter les coefficients : effet et significativité.

Étapes avec vous valeurs scaller et selectionnées :

1. Split : `X_train`, `X_test`, `y_train`, `y_test`.
2. Entraîner une `LogisticRegression()`.
3. Afficher les coefficients et les p-values avec `statsmodels`.
4. Interpréter les coefficients : effet et significativité.

Bloc 4 : Évaluation du modèle de classification

Métriques à calculer :

- Accuracy
- Precision, Recall
- F1-score
- Matrice de confusion
- Tradeoff curve Precision/Recall selon le seuil de décision
- ROC curve + AUC - Expliquer sa signification en détail

Courbes recommandées :

- `plot_confusion_matrix`
- `plot_roc_curve`
- Courbe de precision-recall personnalisée avec seuils

Interprétation du seuil de classification - Threshold - voir notes

1. Utiliser `predict_proba` pour extraire les probabilités.
2. Créer une fonction `custom_predict(X, seuil)`.
3. Visualiser comment le changement de seuil impacte Recall / Precision.
4. Choisir un seuil optimal selon l'objectif

Comparer vos scores entre les deux modèles défini plus haut.

Bloc 5 : Optimisation par Random Search

Éxpliquer le principe du random search en quelques mots.

Dans une régression linéaire sur quoi je peux intervenir pour la partie random search ?

Étapes :

1. Définir une grille de recherche avec `param_distributions` (ex: `C`, `penalty`, `solver`).
2. Lancer un `RandomizedSearchCV` avec `cv=5`, `scoring='f1'`.
3. Afficher les meilleurs paramètres.
4. Comparer les performances du modèle optimisé avec le modèle de base.

Objectifs finaux

- Comprendre les métriques de classification et leurs limites.
- Choisir les variables utiles et les préparer correctement.
- Entraîner et améliorer un modèle de régression logistique.
- Visualiser et interpréter les performances.
- Réaliser une optimisation via RandomSearchCV.

N'oubliez pas de documenter votre notebook avec des commentaires explicatifs à chaque étape !